Problem Statement: Write code for Merge Sort (a divide-and-conquer, stable algorithm with O(n log n) time in all cases).

1. Objectives:

To implement the Merge Sort algorithm, which is a divide-and-conquer, stable algorithm with a time complexity of O (n log n) in all cases, for sorting an unsorted array.

2. Tools & Environment:

1. Language: C++
2. Compiler: g++ (MinGW-w64), version 8.1.0
3. IDE: Visual Studio Code, v1.101.2
4. Operating System: Microsoft Windows 11 Pro
5. OS Version: 10.0.26100 (Build 26100)
6. System Type: x64-based PC
7. Word Processor: Microsoft 365 Apps for enterprise

3. Code Implementation:

Merge_Sort.cpp

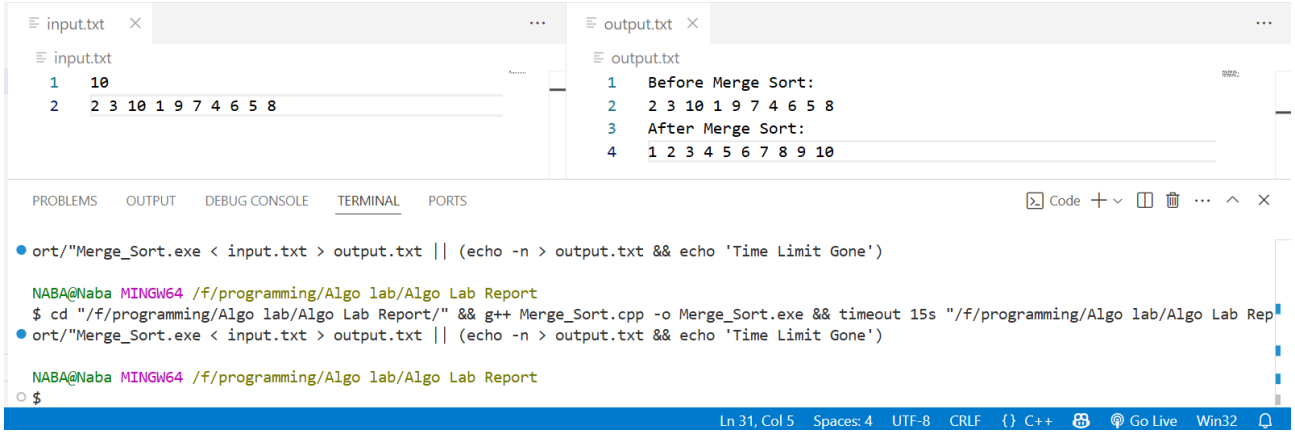
Algo Lab Report > Merge_Sort.cpp > merge(int [], int, int, int)

```cpp
#include<bits/stdc++.h>
using namespace std;

/*
merge function
Merges two subarrays of c[]
-left subarray is c[left to mid]
-right subarray is c[ mid+1 to right]
*/
void merge(int c[], int left, int mid, int right)
{
    // size of left and right subarrays
    int n = mid-left+1;
    int m = right - mid;

    //Create temporary arrays
    int a[n], b[m];

    int k = left;
```

```
    // Copy element from c[] to temporary array a[]
    for(int i = 0; i < n; i++){
        a[i] = c[k++];
    }
    k = mid+1;

    // Copy element from c[] to temporary array b[]
    for(int i = 0; i < m; i++){
        b[i] = c[k++];
    }
    int i = 0, j = 0, current = left;

    //Merge the temporary arrays back into c[]
    while(i < n && j < m)
    {
        if(a[i] < b[j])
            c[current++] = a[i++];
        else
            c[current++] = b[j++];
    }

    //Copy the remaining elements from a to c, if any
    while(i < n)
    {
        c[current++] = a[i++];
    }

    //Copy the remaining elements from b to c, if any
    while(j < m)
    {
        c[current++] = b[j++];
    }
}
```

```
// Recursively sorts the array using merge sort
void mergeSort(int a[], int left, int right)
{
    // base case
    if(left < right)
    {
        //Mid point
        int mid = (left + right)/2;

        // sort the left and right halves
        mergeSort(a, left, mid);
        mergeSort(a, mid+1, right);
        merge(a, left, mid, right);
    }
}

int main(){
    int n;
    cin >> n;

    int a[n];
    //Input array elements
    for(int i =0; i < n; i++)
    {
        cin >> a[i];
    }
    cout << "Before Merge Sort: " <<endl;
    for(int i = 0; i < n; i++)
    {
        cout << a[i] << " ";
    }
    cout << endl;

    // Call the mergeSort function
    mergeSort(a, 0, n-1);

    cout << "After Merge Sort: " <<endl;
    for(int i = 0; i < n; i++)
    {
        cout << a[i] << " ";
    }
    cout << endl;

    return 0;
}
```

4. Results & Output :



5. Code Description & Conclusion:

Code Description:

- This program sorts an array using the Merge Sort algorithm and follows the Divide and Conquer strategy.
- The merge() function merges two sorted halves of the array into one sorted segment.
- The mergeSort() recursively splits the array into halves until each part has only one element, then merges them back in sorted order.
- The main() function takes array input and prints the array before and after sorting.

Conclusion:

Merge Sort has an efficient time complexity of O(n log n). It needs extra space for temporary arrays, so memory usage is higher. The Merge Sort algorithm is a good choice when stability and performance matter.

Problem Statement: Write code for Quick Sort (a divide-and-conquer, in-place but unstable algorithm with average O(n log n) but worst-case O(n 2) time).

1. Objectives:

To implement the Quick Sort algorithm, which is a divide-and-conquer algorithm, a In-place but unstable algorithm, with an average-case time complexity of O(n log n), but worst- case $O(n^2)$ time, for sorting an unsorted array.

2. Tools & Environment:

1. Language: C++
2. Compiler: g++ (MinGW-w64), version 8.1.0
3. IDE: Visual Studio Code, v1.101.2
4. Operating System: Microsoft Windows 11 Pro
5. OS Version: 10.0.26100 (Build 26100)
6. System Type: x64-based PC
7. Word Processor: Microsoft 365 Apps for enterprise

3. Code Implementation:

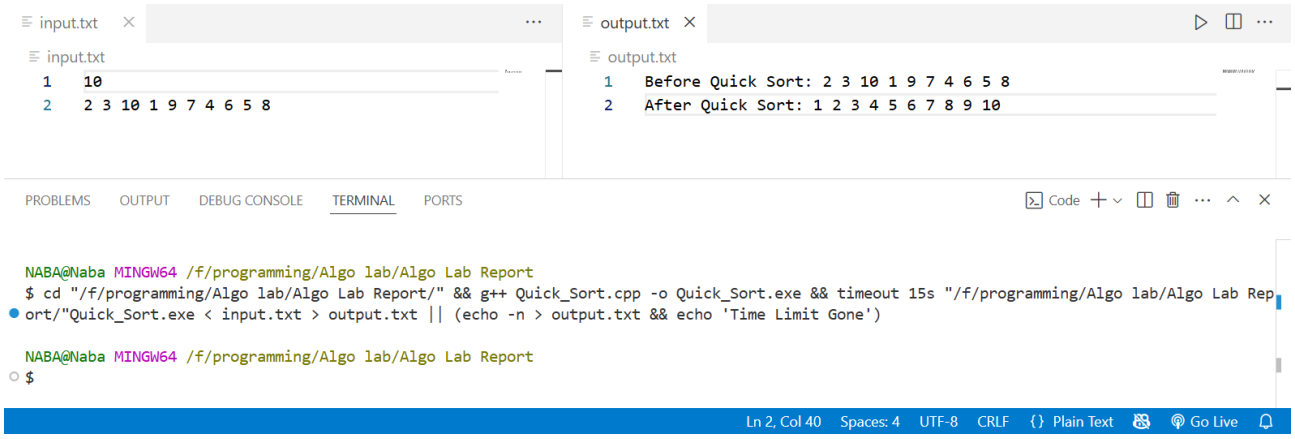
Merge_Sort.cpp Quick_Sort.cpp ×

Algo Lab Report > Quick_Sort.cpp > main()

```cpp
#include<bits/stdc++.h>
using namespace std;

// partition function
int partition(int a[], int left, int right)
{
    // Take the last element as the pivot
    int pivot = a[right];

    // i keeps track of the index of smaller elements
    int i = left - 1;

    for (int j = left; j < right; j++)
    {
        // If a[j] is less than the pivot
        if (a[j] < pivot)
        {
            i++;
            swap(a[i], a[j]);
        }
    }
```

```
    // Place the pivot in the correct position
    swap(a[i + 1], a[right]);

    // Return position of the partition
    return i + 1;
}

// Recursively sorts the array using QuickSort
void QuickSort(int a[], int left, int right)
{
    // base case
    if (left < right)
    {
        // partition index
        int p = partition(a, left, right);
        QuickSort(a, left, p - 1);
        QuickSort(a, p + 1, right);
    }
}

int main()
{
    int n;
    cin >> n;

    int a[n];
    // Input the array elements
    for (int i = 0; i < n; i++){
        cin >> a[i];
    }

    cout << "Before Quick Sort: ";
    for(int i = 0; i < n; i++){
        cout << a[i] << " ";
    }
    cout << endl;

    // Call QuickSort function
    QuickSort(a, 0, n - 1);

    cout << "After Quick Sort: ";
    for(int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }
    cout << endl;
    return 0;
}
```

4.Results & Output :



5. Code Description & Discussion/Conclusion:

Code Description:

- This program sorts an array using the Quick Sort algorithm and follows the Divide and Conquer strategy.
- The partition() function selects a pivot (last element) and rearranges the array so that smaller elements come before the pivot and larger ones after.
- The QuickSort() function recursively sorts the left and right parts of the array using the pivot's position.
- The main() function takes array input and prints the array before and after sorting.

Conclusion:

Quick Sort has an average time complexity of O(n log n), but it can go up to $O(n^2)$ in the worst case.It works in-place without using extra memory, which makes it faster in practice compared to Merge Sort. However, Quick Sort is not stable by default.

Problem Statement: Implement Fractional Knapsack Using Greedy Approach.

1. Objectives:

To implement the Fractional Knapsack Using Greedy Approach and to maximize the total value within the given capacity of the bag by taking full or fractional parts of items.

2. Tools & Environment:

1. Language: C++
2. Compiler: g++ (MinGW-w64), version 8.1.0
3. IDE: Visual Studio Code, v1.101.2
4. Operating System: Microsoft Windows 11 Pro
5. OS Version: 10.0.26100 (Build 26100)
6. System Type: x64-based PC
7. Word Processor: Microsoft 365 Apps for enterprise

3. Code Implementation:

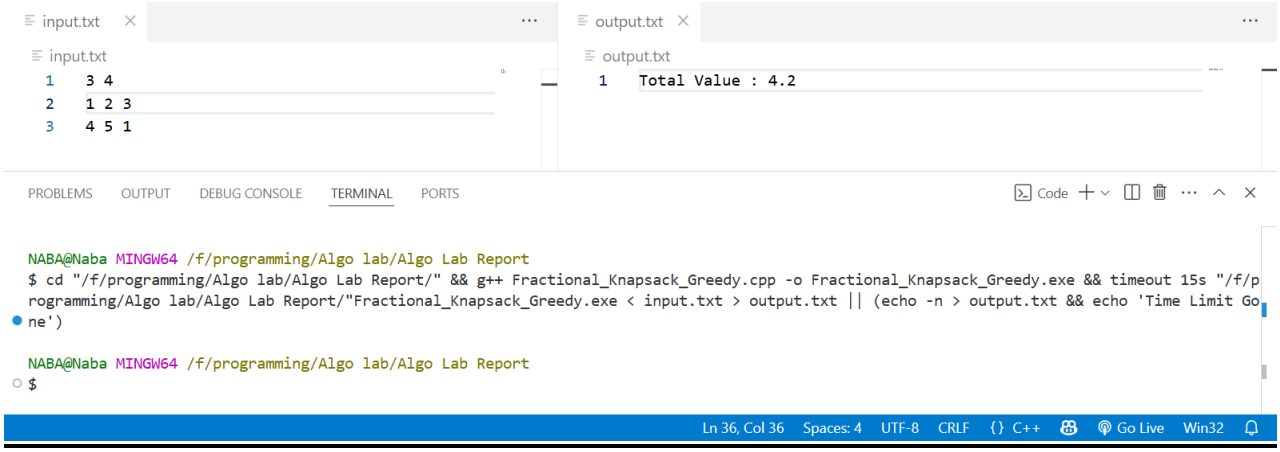
Fractional_Knapsack_Greedy.cpp

Algo Lab Report > Fractional_Knapsack_Greedy.cpp > main()

```cpp
#include<bits/stdc++.h>
using namespace std;

int main()
{
    int n;     // number of items
    cin >> n;
    double C;  // maximum capacity
    cin >> C;

    vector<double> val(n), w(n);

    // Input values of the items in array "val"
    for(int i=0;i<n;i++){
        cin >> val[i];
    }

    // Input weights of the items in array "w"
    for(int i=0;i<n;i++){
        cin >> w[i];
    }

    //Track which items have been used
    vector<bool> used(n, false);

    double curr_weight = C;         // Current remaining capacity
    double total_value = 0.0;       // Initialize total value
```

```
    //Repeat until there is no more capacity
    while (curr_weight > 0)
    {
        int maxi = -1;
        for (int i = 0; i < n; ++i) {
            if (!used[i] && w[i] > 0)
            {
                if (maxi == -1 || ((val[i] / w[i]) > (val[maxi] / w[maxi])))
                {
                    maxi = i;
                }
            }
        }
        // No items left unused, so break the loop
        if (maxi == -1)
        {
            break;
        }

        // Mark the item as taken (fully or partially)
        used[maxi] = true;
        if (w[maxi] <= curr_weight)
        {
            curr_weight -= w[maxi];
            total_value += val[maxi];   // add full value

        }
        else
        {
            double fraction = curr_weight / w[maxi];
            total_value += val[maxi] * fraction;     // add fractional value
            curr_weight = 0;   // No more capacity
        }
    }
    cout << "Total Value : " << total_value <<endl;

    return 0;
}
```

4. Results & Output :



5. Code Description & Discussion/Conclusion:

Code Description:

- This program solves the Fractional Knapsack problem using a greedy approach.\
- It defines a structure to store item values and weights.
- Items are sorted by their value-to-weight ratio in descending order using a custom compare function.
- The program adds full items to the knapsack when possible; otherwise, it adds a fractional part of an item.
- Finally, it calculates and prints the total value.

Discussion/Conclusion:

The greedy approach works well for the fractional knapsack because we can take parts of items. Sorting items by value-to-weight ratio is key. The code is efficient and easy to understand.

Problem Statement: Implement the 0-1 Knapsack Problem Using Dynamic Programming

1. Objectives:

To implement the 0-1 Knapsack problem using the Top-Down Dynamic Programming Approach (memoization) to find the maximize value that can be placed in a bag with capacity C, with a time complexity of O(n*C), where n is the number of items and C is the maximum capacity of the bag.

2. Tools & Environment:

1. Language: C++
2. Compiler: g++ (MinGW-w64), version 8.1.0
3. IDE: Visual Studio Code, v1.101.2
4. Operating System: Microsoft Windows 11 Pro
5. OS Version: 10.0.26100 (Build 26100)
6. System Type: x64-based PC
7. Word Processor: Microsoft 365 Apps for enterprise

3. Code Implementation:

Welcome | 0-1_knapsack_Using_Bottom-Up_DP_Tabulation.cpp

0-1_knapsack_Using_Bottom-Up_DP_Tabulation.cpp > ...

```cpp
#include <bits/stdc++.h>
using namespace std;
vector<int> val, weight;
vector<vector<int>> dp;

// Function to solve 0-1 Knapsack using bottom-up dynamic programming
int knapsack(int n, int W)
{
    // Build dp table iteratively
    for (int i = 0; i <= n; i++)        // For each item count from 0 to n
    {
        for (int j = 0; j <= W; j++)    // For each capacity from 0 to W
        {
            // Base case:
            if (i == 0 || j == 0)
            {
                dp[i][j] = 0;
            }
            else {
                int pick = 0;
                // If current item's weight <= current capacity j
                // We can pick this item and add its value plus best value without this item's weight
                if (weight[i - 1] <= j)
                    pick = val[i - 1] + dp[i - 1][j - weight[i - 1]];

                // Option to NOT pick the current item
                int notPick = dp[i - 1][j];
```

```cpp
                // Take the better of picking or not picking
                dp[i][j] = max(pick, notPick);
            }
        }
    }
    // dp[n][W] contains the answer for n items and capacity W
    return dp[n][W];
}

int main() {
    int n, C;
    cin >> n >> C;   // Input number of items and knapsack capacity

    val.resize(n);      // Resize vectors to hold n elements
    weight.resize(n);
    dp.resize(n + 1, vector<int>(C + 1, 0));  // Initialize DP table with zeros

    // Input values of items
    for (int i = 0; i < n; i++)
        cin >> val[i];

    // Input weights of items
    for (int i = 0; i < n; i++)
        cin >> weight[i];

    // Call knapsack function and print the result
    cout << "Total Value : " << knapsack(n, C) << endl;

    return 0;
}
```

4. Results & Output :

input.txt

```
3 4
1 2 3
4 5 1
```

output.txt

```
Total Value : 3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
NABA@Naba MINGW64 /f/programming/Algo lab/Algo Lab Report
$ cd "/f/programming/Algo lab/Algo Lab Report/" && g++ 0-1_knapsack_Using_Bottom-Up_DP_Tabulation.cpp -o 0-1_knapsack_Using_Bottom-Up_DP_Tabulation.exe && timeout 15s "/f/programming/Algo lab/Algo Lab Report/"0-1_knapsack_Using_Bottom-Up_DP_Tabulation.exe < input.txt > output.txt
ing_Bottom-Up_DP_Tabulation.exe < input.txt > output.txt || (echo -n > output.txt && echo 'Time Limit Gone')

NABA@Naba MINGW64 /f/programming/Algo lab/Algo Lab Report
$
```

Ln 48, Col 23 Spaces: 4 UTF-8 CRLF {} C++ Go Live Win32

5. <u>Code Description & Discussion/Conclusion:</u>

Code Description:

- This program solves the 0-1 Knapsack problem using bottom-up dynamic programming.
- It stores item values and weights in vectors and uses a 2D dp table to keep track of max values for different capacities and item counts.
- The knapsack function fills the table by deciding whether to pick or skip each item to maximize value without exceeding capacity.
- The result is the maximum value possible with all items and the given capacity.

Discussion/Conclusion:

The DP approach efficiently solves 0-1 knapsack with time complexity O(n*C). It doesn't allow fractions of items, unlike fractional knapsack.

Problem Statement: Write a program to perform Breadth-First Search (BFS) on a graph using C/C++/Python.

1. Objectives:

To implement the Breadth-First Search (BFS) algorithm for traversing an undirected graph with a time complexity of O(V+E), where V is the number of vertices and E is the number of edges.

2. Tools & Environment:

1. Language: C++
2. Compiler: g++ (MinGW-w64), version 8.1.0
3. IDE: Visual Studio Code, v1.101.2
4. Operating System: Microsoft Windows 11 Pro
5. OS Version: 10.0.26100 (Build 26100)
6. System Type: x64-based PC
7. Word Processor: Microsoft 365 Apps for enterprise

3. Code Implementation:

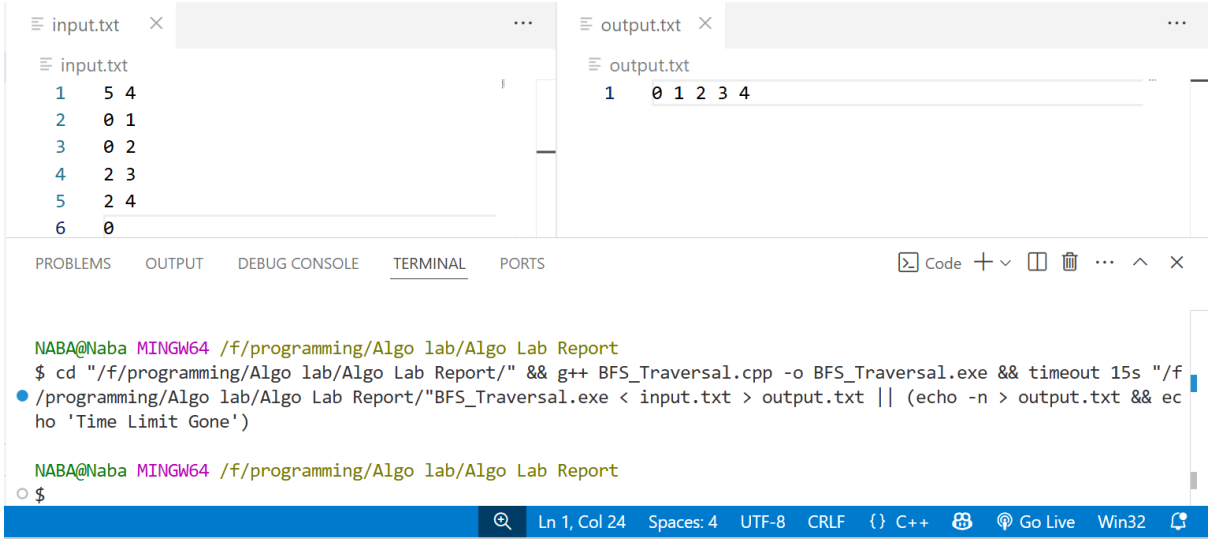
BFS_Traversal.cpp

Algo Lab Report > BFS_Traversal.cpp > bfs(int)

```cpp
#include<bits/stdc++.h>
using namespace std;

// Adjacency list to store an undirected graph
vector<int> adj_list[1005];

// To store the BFS traversal order
vector<int> ans;

// To keep track of visited nodes, create a visited array
bool vis[1005];

// BFS function to traverse the graph
void bfs(int src)
{
    // Create a queue
    queue<int>q;

    // Push the source node into the queue & Mark it as visited
    q.push(src);
    vis[src] = true;

    while (!q.empty())
    {
        // Store front of the queue in a variable and then dequeue it from the queue
        int node = q.front();
        q.pop();
        ans.push_back(node);
```

```
        // Get all unvisited  children of the current node & push them into the queue if not visited
        for(int child : adj_list[node])
        {
            if( !vis[child])
            {
                q.push(child);
                vis[child] = true;  // mark it as visited.
            }
        }

    }
}
int main()
{
    // Input: number of vertices (n) and edges (e)
    int n, e;
    cin >> n >> e;

    // Take the input for the undirected graph using adjacency list
    while(e--)
    {
        int a, b;
        cin >> a >> b;

        adj_list[a].push_back(b);
        adj_list[b].push_back(a);
    }

    // Initialize the visited array as false
    memset(vis, false, sizeof(vis));

    // Call the bfs function
    int src;
    cin >> src;
    bfs(src);

    // print the BFS traversal result
    for(int i = 0; i < ans.size(); i++)
    {
        cout << ans[i] << " ";
    }
    cout << endl;

    return 0;
}
```

4. Results & Output :



5. Code Description & Discussion/Conclusion:

Code Description:

- This program performs a Breadth-First Search (BFS) on an undirected graph.
- It uses an adjacency list to store the graph and a queue to traverse nodes level by level.
- Visited nodes are marked to prevent revisiting.
- It prints the nodes in the order they are visited during BFS.

Conclusion:

BFS explores the graph level-wise. It is helpful for finding the shortest path in unweighted graphs. The time complexity is O(V + E), where V is the number of vertices and E is the number of edges.

Problem Statement: Write a program to perform Depth-First Search (DFS) on a graph using C/C++/Python.

1. Objectives:

To implement the Depth-First Search (DFS) algorithm for traversing an undirected graph with a time complexity of O(V+E), where V is the number of vertices and E is the number of edges.

2. Tools & Environment:

1. Language: C++
2. Compiler: g++ (MinGW-w64), version 8.1.0
3. IDE: Visual Studio Code, v1.101.2
4. Operating System: Microsoft Windows 11 Pro
5. OS Version: 10.0.26100 (Build 26100)
6. System Type: x64-based PC
7. Word Processor: Microsoft 365 Apps for enterprise

3. Code Implementation:

DFS_Traversal.cpp

Algo Lab Report > DFS_Traversal.cpp > main()

```cpp
#include<bits/stdc++.h>
using namespace std;

// Adjacency list to store an undirected graph
vector<int> adj_list[1005];

// To store the DFS traversal order
vector<int> ans;

// To keep track of visited nodes, create a visited array
bool vis[1005];

// DFS function to traverse the graph
void dfs(int src)
{
    //push the current node to the ans vector
    ans.push_back(src);
    vis[src] = true;

    //visit all unvisited adjacent nodes
    for(int child : adj_list[src])
    {
        if(!vis[child])
        {
            dfs(child);
        }
    }
}
```

```cpp
int main()
{
    // Input: number of vertices (n) and edges (e)
    int n, e;
    cin >> n >> e;

    // Take the input for the undirected graph using adjacency list
    while(e--)
    {
        int a, b;
        cin >> a >> b;

        adj_list[a].push_back(b);
        adj_list[b].push_back(a);
    }

    // Initialize the visited array as false
    memset(vis, false, sizeof(vis));

    // Call the DFS function
    int src;
    cin >> src;
    dfs(src);

    // print the DFS traversal result
    for(int i = 0; i < ans.size(); i++)
    {
        cout << ans[i] << " ";
    }
    cout << endl;

    return 0;
}
```

4. Results & Output :

input.txt

```
5 4
0 1
0 2
2 3
2 4
0
```

output.txt

```
0 1 2 3 4
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
NABA@Naba MINGW64 /f/programming/Algo lab/Algo Lab Report
$ cd "/f/programming/Algo lab/Algo Lab Report/" && g++ DFS_Traversal.cpp -o DFS_Traversal.exe && timeout 15s "/f
/programming/Algo lab/Algo Lab Report/"DFS_Traversal.exe < input.txt > output.txt || (echo -n > output.txt && ec
ho 'Time Limit Gone')

NABA@Naba MINGW64 /f/programming/Algo lab/Algo Lab Report
$
```

Ln 7, Col 36 Spaces: 4 UTF-8 CRLF {} C++ Go Live Win32

5. <u>Code Description & Discussion/Conclusion:</u>

Code Description:

- This program performs Depth-First Search (DFS) on an undirected graph.
- It uses recursion to explore nodes as deeply as possible along each path.
- Uses an adjacency list to represent the graph and a visited array to avoid revisiting nodes.
- Prints nodes in the order they are visited during DFS.

Conclusion:

DFS explores the graph depth-first.Useful for detecting cycles, finding connected components, and more.Time complexity is O(V + E), where V is vertices and E is edges.